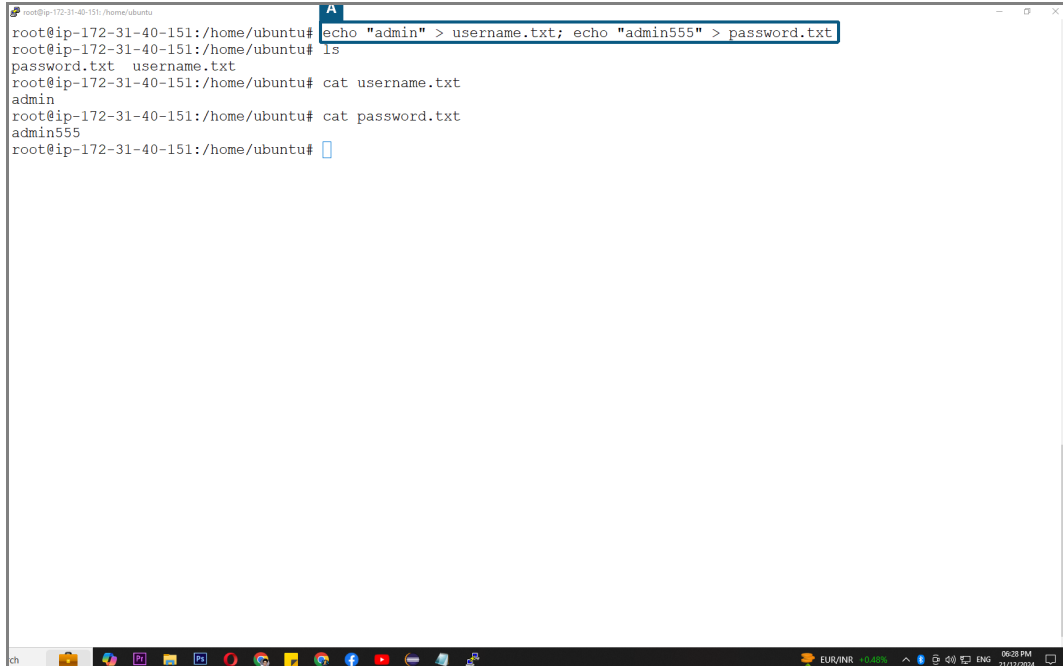

Secret Object In Kubernetes

Rohit Surwade

X

Create dummy username & password files to test the functionality of Secret.



```
root@ip-172-31-40-151: /home/ubuntu# echo "admin" > username.txt; echo "admin555" > password.txt
root@ip-172-31-40-151: /home/ubuntu# ls
password.txt  username.txt
root@ip-172-31-40-151: /home/ubuntu# cat username.txt
admin
root@ip-172-31-40-151: /home/ubuntu# cat password.txt
admin555
root@ip-172-31-40-151: /home/ubuntu#
```

1.

commands:

1. echo "admin" > username.txt;
2. echo "admin555" > password.txt

How to create Secret object and list them.

```
root@ip-172-31-40-151:/home/ubuntu# kubectl create secret generic mysecret --from-file=username.txt --from-file=password.txt
secret/mysecret created
root@ip-172-31-40-151:/home/ubuntu# kubectl get secret
NAME                                TYPE                                DATA  AGE
default-token-2qfs4                kubernetes.io/service-account-token 3      7m33s
mysecret                           Opaque                              2      15s
root@ip-172-31-40-151:/home/ubuntu#
```

2.

commands:

1. `kubectl create secret generic <secret-name> --from-file=filename.`
2. `kubectl get secret.`

Lets try describe command on Secret object.

```
root@ip-172-31-40-151:/home/ubuntu# kubectl create secret generic mysecret --from-file=username.txt --from-file=password.txt
secret/mysecret created
root@ip-172-31-40-151:/home/ubuntu# kubectl get secret
NAME                                TYPE                                DATA  AGE
default-token-2qfs4                kubernetes.io/service-account-token 3      7m33s
secret                             Opaque                              2      15s
root@ip-172-31-40-151:/home/ubuntu# kubectl describe secret mysecret
Name:         mysecret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type: Opaque

Data
====
password.txt:  9 bytes
username.txt:  6 bytes
root@ip-172-31-40-151:/home/ubuntu#
```

3.

WE WON'T BE ABLE TO SEE WHAT'S INSIDE THE FILES

A manifest the create a POD and consume the Secret(Volume mount)

```
root@ip-172-31-40-131: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: myvolsecret
spec:
  containers:
  - name: c1
    image: centos
    command: ["/bin/bash", "-c", "while true; do echo helloo-XD; sleep 5; done"]
    volumeMounts:
    - name: testsecret
      mountPath: "/tmp/mysecrets" # Secret files will be mounted here as ReadOnly by default
  volumes:
  - name: testsecret
    secret:
      secretName: mysecret # Ensure this matches the name of an existing Secret

-- INSERT --
```

4.

(This Kubernetes manifest defines a Pod named myvolsecret with a single container named c1 running the centos image.

The container executes a command that continuously prints "helloo-XD" every 5 seconds. A volume named testsecret is mounted at

/tmp/mysecrets in the container, and it is backed by a Secret named mysecret.

All key-value pairs in the mysecret Secret are mounted as

individual files in the /tmp/mysecrets directory, where each key becomes a filename and its corresponding value becomes the file content.

The type of Secret consumption in this setup is mounted as a volume, making it suitable for applications that need to access sensitive data

(e.g., credentials, API keys) in file format. Ensure the mysecret Secret exists with the required key-value pairs before applying this manifest.

After deployment, the container will have access to the Secret's content under /tmp/mysecrets, and the logs will reflect the updated output "helloo-XD".)

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: myvolsecret
```

```
spec:
```

```
  containers:
```

```
  - name: c1
```

```
    image: centos
```

```
    command: ["/bin/bash", "-c", "while true; do echo helloo-XD; sleep 5; done"]
```

```
    volumeMounts:
```

```
    - name: testsecret
```

```
      mountPath: "/tmp/mysecrets" # Secret files will be mounted here as ReadOnly by default
```

```
  volumes:
```

```
  - name: testsecret
```

```
    secret:
```

```
      secretName: mysecret # Ensure this matches the name of an existing Secret
```

Applying the Manifest (our pod is created successfully)

```
root@ip-172-31-40-151:/home/ubuntu# B deploysecret.yml
root@ip-172-31-40-151:/home/ubuntu# kubectl apply -f deploysecret.yml
pod/myvolsecret created
root@ip-172-31-40-151:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myvolsecret   0/1     ContainerCreating   0           9s
root@ip-172-31-40-151:/home/ubuntu# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
myvolsecret   1/1     Running    0           15s
root@ip-172-31-40-151:/home/ubuntu#
```

5.

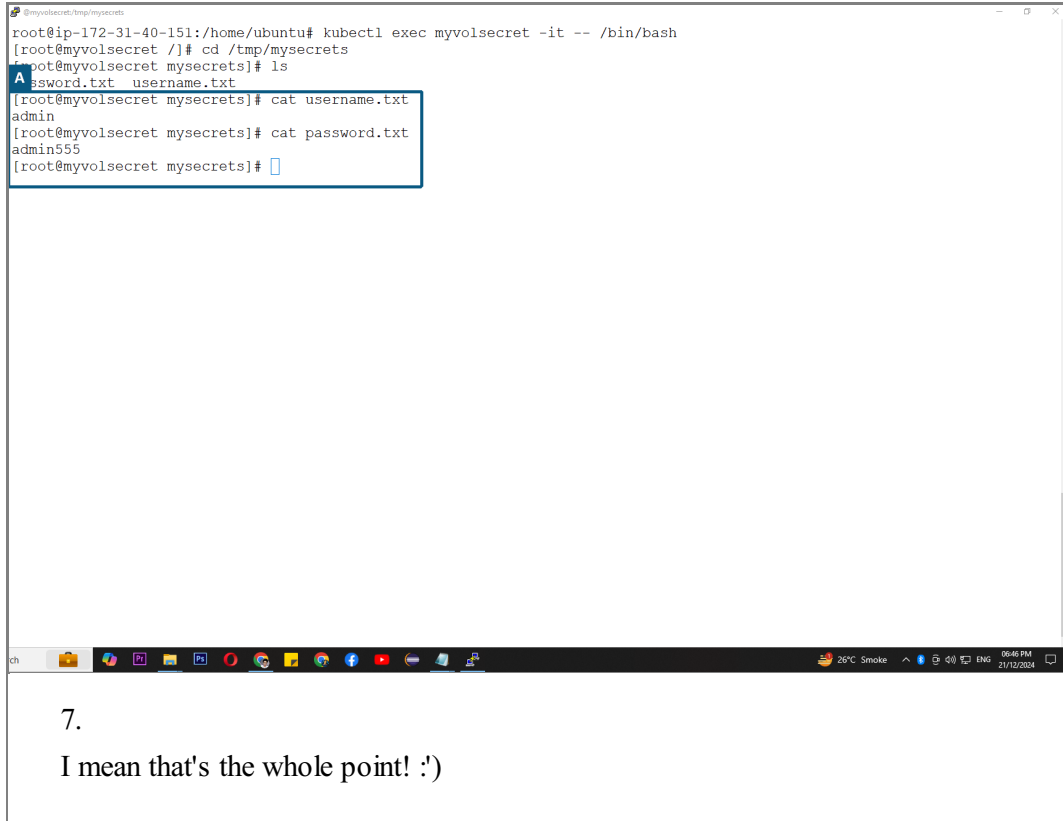
Enter the container and verify if the Secret is reflecting

```
root@ip-172-31-40-151:/home/ubuntu# A kubectl exec myvolsecret -it -- /bin/bash
root@myvolsecret /# B cd /tmp/mysecrets
root@myvolsecret mysecrets# ls
password.txt  username.txt
root@myvolsecret mysecrets#
```

6.

As per the instructions given in the manifest Secret is reflecting inside the container at the mounted path i.e /tmp/mysecrets

We can access the Secret files as they are being consumed by the POD!



```
@myvolsecret/tmp/mysecrets
root@ip-172-31-40-151:/home/ubuntu# kubectl exec myvolsecret -it -- /bin/bash
[root@myvolsecret /]# cd /tmp/mysecrets
[root@myvolsecret mysecrets]# ls
password.txt  username.txt
[root@myvolsecret mysecrets]# cat username.txt
admin
[root@myvolsecret mysecrets]# cat password.txt
admin555
[root@myvolsecret mysecrets]#
```

7.

I mean that's the whole point! :')